

Tensoren und Tensorprodukte

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Lineare Algebra**

- Vektorräume und Unterräume
- Basis und Dimension
- Lineare Abbildungen und Multilinearität
- Äußeres Produkt: $\boldsymbol{v} \otimes \boldsymbol{w} = \boldsymbol{vw}^T$
- $\text{span}(\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_n\}) = \{\boldsymbol{w} : \boldsymbol{w} = \sum_i c_i \boldsymbol{v}_i; c_i \in \mathbb{R}\}$

- **Analysis**

- Multivariate Funktionen $f(\boldsymbol{x}) = f(x_1, x_2, \dots, x_n)$

- **Gleichungssysteme** (Kapitel 5): LGS $\boldsymbol{M}\boldsymbol{z} = \boldsymbol{c}$ lösen (LU, Cholesky)

Multilineare Abbildungen

Multilineare Abbildungen

- Bisher wurde ein Vektor $\boldsymbol{v} \in V$ linear auf einen Vektor $\boldsymbol{w} \in W$ abgebildet.
- Jetzt werden Paare oder Tupel von Vektoren $(\boldsymbol{v}_1, \dots, \boldsymbol{v}_n) \in V_1 \times \dots \times V_n$ “linear” auf ein $\boldsymbol{w} \in W$ abgebildet,
 - $\rightarrow$ Wir sprechen von einer *multilinearen Abbildung*.
- Linearität bezieht sich hier auf die *einzelnen* Argumente der Abbildung.

Multilineare Abbildungen

Def.: Multilineare Abbildung

Seien $V_1, \dots, V_n, W$ Vektorräume. Eine **multilineare Abbildung** ist eine Abbildung $f: V_1 \times \dots \times V_n \rightarrow W$, sodass:

$$\begin{aligned} & f(v_1, \dots, cv_i + c'v'_i, \dots, v_n) \\ &= cf(v_1, \dots, v_i, \dots, v_n) + c'f(v_1, \dots, v'_i, \dots, v_n) \end{aligned}$$

für alle $i = 1, \dots, n, c, c' \in \mathbb{R}, v_i, v'_i \in V_i$ und $v_\ell \in V_\ell$ ($\ell \neq i$).

Anmerkungen:

- Im Spezialfall $n = 2$ nennen wir die Abbildung **bilinear**.
- **Achtung:** **Multilineare Abbildungen** sind in der Regel nicht linear!

Intuition: Linear vs. Multilinear

Fläche eines Rechtecks als Funktion seiner Seitenlängen:

$$f: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}, \quad (x, y) \mapsto xy$$

Warum ist f nicht linear?

- Für $c = 2$: $f(2x, 2y) = 2x \cdot 2y = 4xy = 4f(x, y) \neq 2f(x, y)$ (für $xy \neq 0$)
- Linear wäre: $f(cx, cy) = c \cdot f(x, y)$

Aber f ist bilinear:

$$y \text{ konstant: } f(x_1 + x_2, y) = (x_1 + x_2) \cdot y = x_1y + x_2y = f(x_1, y) + f(x_2, y)$$

$$f(cx, y) = (cx) \cdot y = c(xy) = cf(x, y)$$

$$x \text{ konstant: } f(x, y_1 + y_2) = x \cdot (y_1 + y_2) = xy_1 + xy_2 = f(x, y_1) + f(x, y_2)$$

$$f(x, cy) = x \cdot (cy) = c(xy) = cf(x, y)$$

$\implies$ Die Funktion ist **in jedem Argument einzeln linear!**

Multilineare Abbildungen

Beispiel:

- Multiplikation von zwei reellen Zahlen, $f: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}, (x_1, x_2) \mapsto x_1 x_2$, ist bilinear (s.a. Bsp. zur Fläche)

Beispiel:

- Jedes Skalarprodukt $f: V \times V \rightarrow \mathbb{R}, (\mathbf{v}_1, \mathbf{v}_2) \mapsto \langle \mathbf{v}_1, \mathbf{v}_2 \rangle$ ist bilinear.
- Das folgt aus Distributivgesetz, Homogenität und Symmetrie:
$$\langle c\mathbf{v} + c'\mathbf{v}', \mathbf{w} \rangle = \langle c\mathbf{v}, \mathbf{w} \rangle + \langle c'\mathbf{v}', \mathbf{w} \rangle = c\langle \mathbf{v}, \mathbf{w} \rangle + c'\langle \mathbf{v}', \mathbf{w} \rangle$$

Beispiel:

- Die Matrizenmultiplikation $\mathbb{R}^{m \times k} \times \mathbb{R}^{k \times n} \rightarrow \mathbb{R}^{m \times n}$ mit $(\mathbf{A}, \mathbf{B}) \mapsto \mathbf{AB}$ ist bilinear.

Quiz

TRUE/FALSE?

Für jedes multilineare f gilt $f(\mathbf{v}_1, \dots, \mathbf{v}_n) = \mathbf{0}$ sobald $\mathbf{v}_i = \mathbf{0}$ für ein i .

Darstellung von multilinearen Abbildungen

Theorem

Eine Abbildung $T: \mathbb{R}^{n_1} \times \cdots \times \mathbb{R}^{n_k} \rightarrow \mathbb{R}^m$ ist genau dann multilinear, wenn es reelle Koeffizienten

$$A = (a_{i_1, \dots, i_k, j} : 1 \leq i_1 \leq n_1, \dots, 1 \leq i_k \leq n_k, 1 \leq j \leq m)$$

gibt, sodass

$$T(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}) = \sum_{i_1=1}^{n_1} \cdots \sum_{i_k=1}^{n_k} \sum_{j=1}^m a_{i_1, \dots, i_k, j} x_{i_1}^{(1)} \cdots x_{i_k}^{(k)} \mathbf{e}_j,$$

wobei $\mathbf{e}_j$ der j -te Einheitsvektor des $\mathbb{R}^m$ ist. Die Koeffizienten $a_{i_1, \dots, i_k, j}$ sind eindeutig.

Tensoren

Tensoren

- Setzen wir im vorherigen Theorem $k = 1$, erhalten wir Koeffizienten $A = (a_{i,j}: 1 \leq i \leq n, 1 \leq j \leq m)$, sodass

$$T(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^m a_{i,j} x_i \mathbf{e}_j = \sum_{i=1}^n x_i \mathbf{a}_i = \mathbf{A} \mathbf{x},$$

mit $\mathbf{a}_i = (a_{i,1}, \dots, a_{i,m})^\top \in \mathbb{R}^m$ und $\mathbf{A} = (\mathbf{a}_1, \dots, \mathbf{a}_n) \in \mathbb{R}^{m \times n}$, d.h. $(\mathbf{A})_{j,i} = a_{i,j}$.

- Die Ansammlung $\mathbf{A}$ aus nm Koeffizienten nennen wir **Matrix**.
- Sie hat zwei “Dimensionen” weil sie durch zwei Subskripte i und j indiziert wird.
- Allgemeiner beschreibt jede solche Ansammlung von Koeffizienten mit $k + 1$ Indizes (bei $m = 1$ also mit k) eine entsprechende multilineare Funktion

Bsp: Bilineare Abbildung ($k = 2, n_1 = n_2 = 2, m = 1$)

$T: \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ mit $T(\mathbf{x}, \mathbf{y}) = 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2$

Koeffizienten ($a_{i_1, i_2}: 1 \leq i_1, i_2 \leq 2$):

$$a_{1,1} = 2, \quad a_{1,2} = 3, \quad a_{2,1} = -1, \quad a_{2,2} = 4 \implies \mathbf{A} = \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix}$$

Verifikation:

$$\begin{aligned} T(\mathbf{x}, \mathbf{y}) &= \sum_{i_1=1}^2 \sum_{i_2=1}^2 a_{i_1, i_2} x_{i_1} y_{i_2} = a_{1,1}x_1y_1 + a_{1,2}x_1y_2 + a_{2,1}x_2y_1 + a_{2,2}x_2y_2 \\ &= 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2 \quad \checkmark \end{aligned}$$

Numerisches Beispiel:

$$\text{Für } \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \mathbf{y} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}: T(\mathbf{x}, \mathbf{y}) = 2 \cdot 1 \cdot 3 + 3 \cdot 1 \cdot 4 - 1 \cdot 2 \cdot 3 + 4 \cdot 2 \cdot 4 = 6 + 12 - 6 + 32 = 44$$

Alternativ in Tensorschreibweise:

$$T(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{A} \mathbf{y} = \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 & 11 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 44$$

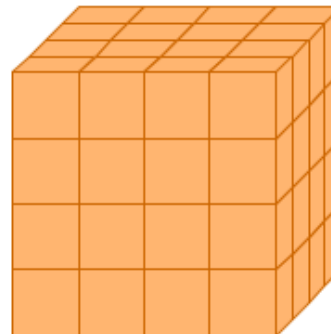
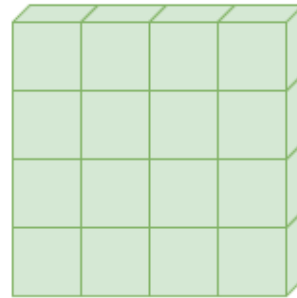
Definition: Tensor

Def.: Tensor

Ein **Tensor** ist eine durch k Indizes indizierte Ansammlung von reellen Zahlen

$$A = (a_{i_1, \dots, i_k} : 1 \leq i_1 \leq n_1, \dots, 1 \leq i_k \leq n_k) \in \mathbb{R}^{n_1 \times \dots \times n_k}.$$

Die Zahl k wird auch **Stufe** des Tensors genannt.



Eine (4×4) -Matrix besteht aus $4^2 = 16$ Zahlen (links); ein $(4 \times 4 \times 4)$ -Tensor besteht aus $4^3 = 64$ Zahlen (rechts). (Quelle: Deisenroth/Faisal/Ong, Fig 5.13)

- Die Menge $\mathbb{R}^{n_1 \times \cdots \times n_k}$ aller $(n_1 \times \cdots \times n_k)$ -Tensoren mit koeffizientenweiser Addition und Multiplikation mit Skalaren ist ein Vektorraum.

Anwendung: Tensoren in Deep Learning

- Eine Matrix stellt z.B. ein (Graustufen-)Bild dar: Dimension 1 indiziert die Pixel vertikal (Höhe), Dimension 2 horizontal (Breite).
- **RGB-Bild:** dritte Dimension indiziert die Farbkanäle $\implies$ Stufe-3-Tensor $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 3}$ ($\approx 150 \cdot 10^3$ Werte pro Bild)
- **Batch/Video:** vierte Dimension indiziert mehrere Bilder — z.B. $\stackrel{\sim}{=} 3$ Kanal-Matrizen verarbeitet ein CNN 32 Bilder gleichzeitig $\implies$ Stufe-4-Tensor $\mathbf{B} \in \mathbb{R}^{32 \times 224 \times 224 \times 3}$ ($\approx 4.8 \cdot 10^6$ Werte)
- **Feature Maps:** Convolutional Layers bilden Tensoren auf Tensoren ab: $c: \mathbb{R}^{32 \times 224 \times 224 \times 3} \rightarrow \mathbb{R}^{32 \times 56 \times 56 \times 64}$
- Rechnungen auf Tensoren können durch spezielle Hardware besonders schnell durchgeführt werden (Google TPUs, NVIDIA Tensor Cores, ...) — essentiell für KI.



(Details & Feature-Map-Visualisierung $\rightarrow$ Anhang)

Produkt von Tensoren

Produkt von Tensoren

Def.: Äußeres Produkt

Das **äußere Produkt** von zwei Vektoren $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ ist definiert als

$$\mathbf{v} \otimes \mathbf{w} := \mathbf{v}\mathbf{w}^\top = \begin{pmatrix} v_1 w_1 & \cdots & v_1 w_n \\ \vdots & & \vdots \\ v_m w_1 & \cdots & v_m w_n \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

- **Äußeres Produkt**: $\otimes: \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$
- vgl Skalarprodukt (“inneres Produkt”): $\langle \cdot, \cdot \rangle: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R} \quad \langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^\top \mathbf{w}$

Äußeres Produkt: Beispiel & Eigenschaften

Beispiel:

$$\mathbf{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{und} \quad \mathbf{w} = \begin{pmatrix} -2 \\ 3 \\ -11 \end{pmatrix}$$

$$\begin{aligned} \mathbf{v} \otimes \mathbf{w} &= \mathbf{v} \mathbf{w}^\top = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} -2 & 3 & -11 \end{pmatrix} \\ &= \begin{pmatrix} -2 & 3 & -11 \\ -4 & 6 & -22 \end{pmatrix} \end{aligned}$$

- Alle Spalten sind Vielfache von $\mathbf{v}$, alle Zeilen sind Vielfache von $\mathbf{w}^\top$
 - Rang ist 1, sofern $\mathbf{v} \neq \mathbf{0}$ und $\mathbf{w} \neq \mathbf{0}$
- Was für eine lineare Abbildung ist $\mathbf{x} \rightarrow (\mathbf{v} \otimes \mathbf{w})\mathbf{x}$? (im Folgenden: $\mathbf{v} \neq \mathbf{0}, \mathbf{w} \neq \mathbf{0}$)
 - $(\mathbf{v} \otimes \mathbf{w})\mathbf{x} = \mathbf{v} \mathbf{w}^\top \mathbf{x} = \mathbf{v} \langle \mathbf{w}, \mathbf{x} \rangle$:
Vektor $\mathbf{v}$ skaliert mit Skalarprodukt von $\mathbf{w}$ und $\mathbf{x}$
 - $\text{col}(\mathbf{v} \otimes \mathbf{w}) = \text{span}(\mathbf{v})$: Bild ist eindimensional
 - $\text{ker}(\mathbf{v} \otimes \mathbf{w}) = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} = 0\}$: Hyperebene orthogonal zu $\mathbf{w}$

Anwendungsbeispiele:

- **SVD**: $\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top = \sum_{i=1}^r \sigma_i \mathbf{u}_i \otimes \mathbf{v}_i$ (Darstellung einer Matrix als gewichtete Summe von normierten Rang-1-Matrizen)
- Statistik: $\text{Cov}(\mathbf{x}) = \mathbb{E} \left[(\mathbf{x} - \mathbb{E}[\mathbf{x}]) (\mathbf{x} - \mathbb{E}[\mathbf{x}])^\top \right] = \mathbb{E} [\mathbf{x} \otimes \mathbf{x}] - \mathbb{E} [\mathbf{x}] \otimes \mathbb{E} [\mathbf{x}]$
- KI: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{Q} \mathbf{K}^\top \right) \mathbf{V}$

Tensorprodukt

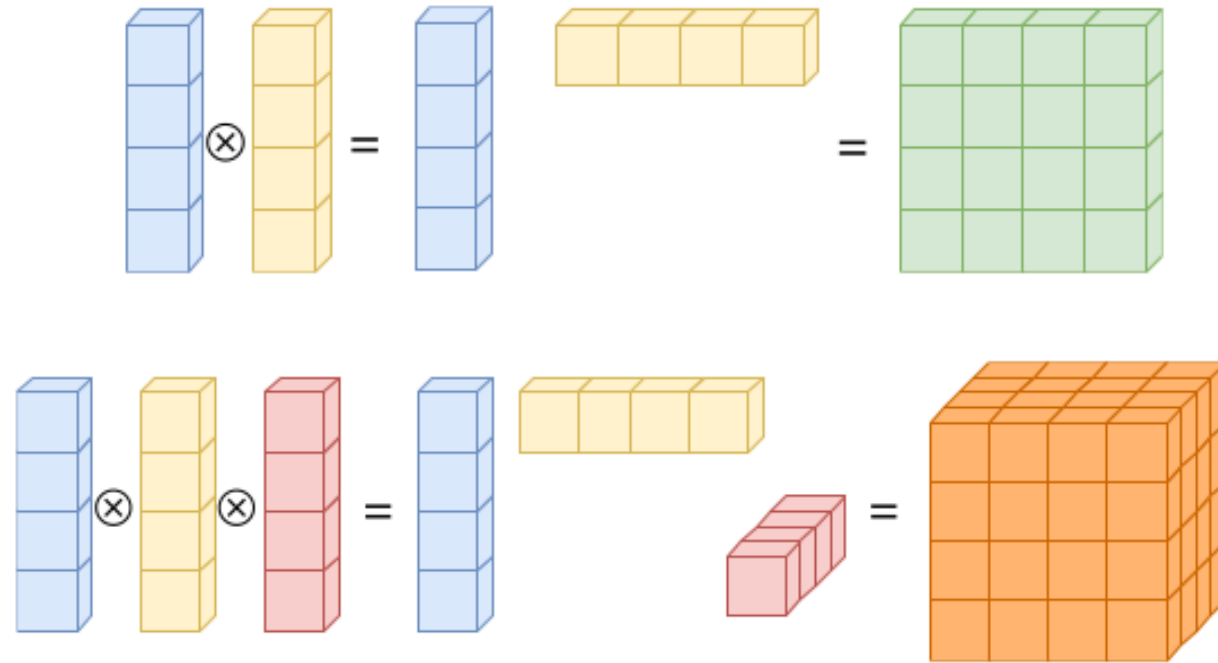
Def.: Tensorprodukt

Das **Tensorprodukt** $\otimes: \mathbb{R}^{m_1 \times \dots \times m_p} \times \mathbb{R}^{n_1 \times \dots \times n_q} \rightarrow \mathbb{R}^{m_1 \times \dots \times m_p \times n_1 \times \dots \times n_q}$ von zwei Tensoren $\mathbf{A} \in \mathbb{R}^{m_1 \times \dots \times m_p}$ und $\mathbf{B} \in \mathbb{R}^{n_1 \times \dots \times n_q}$ ist definiert als der Tensor $\mathbf{C} \in \mathbb{R}^{m_1 \times \dots \times m_p \times n_1 \times \dots \times n_q}$ mit

$$c_{i_1, \dots, i_p, j_1, \dots, j_q} = a_{i_1, \dots, i_p} b_{j_1, \dots, j_q}.$$

- **Äußeres Produkt** ist **Tensorprodukt** zweier Stufe-1-Tensoren ($p = q = 1$, also Vektoren)
- Das **Tensorprodukt** ist bilinear

Produkte von Tensoren



Oben: Das Produkt von zwei Stufe-1-Tensoren ergibt einen Stufe-2-Tensor. Unten: Das Produkt von drei Stufe-1-Tensoren ergibt einen Stufe-3-Tensor. Quelle: Deisenroth/Faisal/Ong, Fig 5.13

Bsp: Tensorprodukt

$$\mathbf{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^2, \quad \mathbf{v} = \begin{pmatrix} 3 \\ 5 \end{pmatrix} \in \mathbb{R}^2, \quad \mathbf{w} = \begin{pmatrix} 7 \\ 11 \end{pmatrix} \in \mathbb{R}^2$$

Tensorprodukt $\mathbf{T} = \mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w} \in \mathbb{R}^{2 \times 2 \times 2}$ hat Einträge $T_{i,j,k} = u_i v_j w_k$:

$$\text{Slice 1 (} k = 1, \text{ also } w_k = w_1 = 7\text{): } T_{1,1,1} = 1 \cdot 3 \cdot 7 = 21, \quad T_{1,2,1} = 1 \cdot 5 \cdot 7 = 35$$

$$T_{2,1,1} = 2 \cdot 3 \cdot 7 = 42, \quad T_{2,2,1} = 2 \cdot 5 \cdot 7 = 70$$

$$\text{Slice 2 (} k = 2, \text{ also } w_k = w_2 = 11\text{): } T_{1,1,2} = 1 \cdot 3 \cdot 11 = 33, \quad T_{1,2,2} = 1 \cdot 5 \cdot 11 = 55$$

$$T_{2,1,2} = 2 \cdot 3 \cdot 11 = 66, \quad T_{2,2,2} = 2 \cdot 5 \cdot 11 = 110$$

$$\implies \mathbf{T}_{\cdot,\cdot,1} = \begin{pmatrix} 21 & 35 \\ 42 & 70 \end{pmatrix}; \quad \mathbf{T}_{\cdot,\cdot,2} = \begin{pmatrix} 33 & 55 \\ 66 & 110 \end{pmatrix}$$

Jede *slice* (Stufe-2-Tensor) dieses Stufe-3-Tensors ist eine Rang-1-Matrix:

- $\mathbf{T}_{\cdot,\cdot,1} = w_1 \cdot (\mathbf{u} \otimes \mathbf{v})$ und $\mathbf{T}_{\cdot,\cdot,2} = w_2 \cdot (\mathbf{u} \otimes \mathbf{v})$

Allgemein:

Ein Tensor der Form $\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}$ hat *Rang 1* (für $\mathbf{u}, \mathbf{v}, \mathbf{w} \neq \mathbf{0}$).

Kroneckerprodukt

Def.: Kroneckerprodukt

Das **Kroneckerprodukt** von zwei Matrizen $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{B} \in \mathbb{R}^{p \times q}$ ist definiert als

$$\begin{aligned} \mathbf{A} \otimes_{\mathbb{K}} \mathbf{B} &:= \begin{pmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \vdots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}b_{11} & \cdots & a_{11}b_{1q} & \cdots & a_{1n}b_{11} & \cdots & a_{1n}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{11}b_{p1} & \cdots & a_{11}b_{pq} & \cdots & a_{1n}b_{p1} & \cdots & a_{1n}b_{pq} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{11} & \cdots & a_{m1}b_{1q} & \cdots & a_{mn}b_{11} & \cdots & a_{mn}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{p1} & \cdots & a_{m1}b_{pq} & \cdots & a_{mn}b_{p1} & \cdots & a_{mn}b_{pq} \end{pmatrix} \in \mathbb{R}^{mp \times nq}. \end{aligned}$$

Obacht – Notation:

$\otimes$ ist oft doppelt belegt (Tensorprodukt und Kroneckerprodukt) – wir schreiben ab hier $\otimes_{\mathbb{K}}$ fürs **Kroneckerprodukt**.

Kroneckerprodukt vs. Tensorprodukt

Kroneckerprodukt ist Variante des Tensorprodukts für Matrizen – **dasselbe Zahlenmaterial, andere Anordnung:**

Für $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{p \times q}$

- **Tensorprodukt** ergibt hier **Stufe-4-Tensor**

$$C = A \otimes B \in \mathbb{R}^{m \times n \times p \times q} \quad \text{mit } c_{i_1, i_2, j_1, j_2} = a_{i_1, i_2} b_{j_1, j_2}$$

- **Kroneckerprodukt** ergibt **Matrix**

$$D = A \otimes_K B \in \mathbb{R}^{mp \times nq} \quad \text{mit } d_{(i_1-1)p+j_1, (i_2-1)q+j_2} = a_{i_1, i_2} b_{j_1, j_2}$$

- C und D unterscheiden sich (nur) darin wie ihre Einträge angeordnet sind – D ist “abgeflachte” Variante von C

Notation:

- In der Literatur steht meist $\otimes$ für *beides* (!); welche Bedeutung gemeint ist, verrät dort das **Format** des Ergebnisses.
- In FMM ist $\otimes_K$ immer das **Kroneckerprodukt**
- $\otimes$ bleibt **äußerem Produkt** bzw. **Tensorprodukt** vorbehalten.

Bsp: Kroneckerprodukt

Beispiel:

- Für

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 2 & 5 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 0 & -1 \end{pmatrix}$$

- Gilt

$$\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B} = \begin{pmatrix} 1 \cdot \mathbf{B} & 0 \cdot \mathbf{B} \\ 2 \cdot \mathbf{B} & 5 \cdot \mathbf{B} \end{pmatrix} = \begin{pmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ -1 & 0 & -1 & 0 & 0 & 0 \\ 6 & 0 & 0 & 15 & 0 & 0 \\ 0 & 4 & 0 & 0 & 10 & 0 \\ -2 & 0 & -2 & -5 & 0 & -5 \end{pmatrix}$$

Vorsicht: $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B} \neq \mathbf{B} \otimes_{\mathbb{K}} \mathbf{A}$! Aber: $(\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B})^{\top} = \mathbf{A}^{\top} \otimes_{\mathbb{K}} \mathbf{B}^{\top}$ – Transponieren dreht die Reihenfolge **nicht** um.

Quiz

Frage 1: Sei $\mathbf{A} \in \mathbb{R}^{3 \times 2}$ und $\mathbf{B} \in \mathbb{R}^{2 \times 4}$. In welchem Raum liegt das **Kroneckerprodukt** $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}$?

1. $\mathbb{R}^{6 \times 8}$
2. $\mathbb{R}^{5 \times 6}$
3. $\mathbb{R}^{3 \times 2 \times 2 \times 4}$
4. $\mathbb{R}^{8 \times 6}$

Frage 2: Sei $\mathbf{S} \in \mathbb{R}^{p \times p}$ eine Matrix mit Einträgen s_{ij} . Wie sieht das Kroneckerprodukt $\mathbf{S} \otimes_{\mathbb{K}} \mathbf{I}_n$ aus?

Anwendung: Blockdiagonale Matrizen

Kroneckerprodukt um Blockdiagonalmatrizen zu erzeugen:

$$\mathbf{I}_n \otimes_{\mathbf{K}} \underbrace{\mathbf{S}}_{p \times p} = \underbrace{\begin{pmatrix} \mathbf{S} & & \\ & \mathbf{S} & \\ & & \ddots \\ & & & \mathbf{S} \end{pmatrix}}_{np \times np}$$

- Nützlich z.B. in Statistik für Kovarianzmatrizen von Vektoren mit unabhängigen Blöcken mit identischer Kovarianz
- **Achtung:**
 $\mathbf{I}_n \otimes_{\mathbf{K}} \mathbf{S}$ ist blockdiagonal, $\mathbf{S} \otimes_{\mathbf{K}} \mathbf{I}_n$ ist es i.A. *nicht* — auf die **Reihenfolge der Faktoren** kommt es an!

Anwendung: Separierbare Kovarianz

- z.B. Daten an m verschiedenen *Orten* zu n verschiedenen *Zeitpunkten* (Annahme hier: Daten geordnet Ort 1 Zeit 1, Ort 2 Zeit 1, Ort 1 Zeit 2, Ort 2 Zeit 2)
- **Modellannahme: Separierbare** Kovarianz mit $\Sigma = \Sigma_T \otimes_K \Sigma_S \in \mathbb{R}^{mn \times mn}$, wobei
 - $\Sigma_T \in \mathbb{R}^{n \times n}$: Kovarianzmatrix zwischen Zeitpunkten
 - $\Sigma_S \in \mathbb{R}^{m \times m}$: Kovarianzmatrix zwischen Orten
 - **Vorteile:** Parametersparsam: $mn(mn + 1)/2$ vs. $m(m + 1)/2 + n(n + 1)/2$; einfachere Interpretation

Beispiel $m = 2$ Orte, $n = 2$ Zeitpunkte, $\Sigma_T = \begin{pmatrix} 1.0 & 0.8 \\ 0.8 & 1.0 \end{pmatrix}$, $\Sigma_S = \begin{pmatrix} 2.0 & 0.5 \\ 0.5 & 2.0 \end{pmatrix}$:

$$\Sigma = \Sigma_T \otimes_K \Sigma_S = \begin{pmatrix} 1.0 \cdot \Sigma_S & 0.8 \cdot \Sigma_S \\ 0.8 \cdot \Sigma_S & 1.0 \cdot \Sigma_S \end{pmatrix} = \begin{pmatrix} 2.0 & 0.5 & 1.6 & 0.4 \\ 0.5 & 2.0 & 0.4 & 1.6 \\ 1.6 & 0.4 & 2.0 & 0.5 \\ 0.4 & 1.6 & 0.5 & 2.0 \end{pmatrix} \in \mathbb{R}^{4 \times 4}$$

Interpretation:

Eintrag (1, 1): Varianz in Ort 1 zu Zeit 1 = 2.0; Eintrag (1, 3): Kovarianz zwischen Ort 1, Zeit 1 und Ort 1, Zeit 2 = $1.6 = 0.8 \cdot 2.0$

Anwendung: vec-Trick — Matrixgleichungen als LGS

Für $\mathbf{X} \in \mathbb{R}^{m \times n}$ stapelt $\text{vec}(\mathbf{X}) = (x_{11}, \dots, x_{m1}, x_{12}, \dots, x_{mn})^\top \in \mathbb{R}^{mn}$ die Spalten von $\mathbf{X}$ zu einem Vektor.

vec-Trick

Für $\mathbf{A} \in \mathbb{R}^{k \times m}$, $\mathbf{X} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times l}$ gilt

$$\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X}).$$

- Macht lineare **Matrix**gleichungen zu gewöhnlichen **LGS** $\mathbf{M}\mathbf{z} = \mathbf{c} \rightarrow$ lösbar mit den Methoden aus Kapitel *Gleichungssysteme* (LU, Cholesky, ...)!
- Wichtige Spezialfälle ($\mathbf{I}_n$: Einheitsmatrix):

$$\text{vec}(\mathbf{AX}) = (\mathbf{I}_n \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X}), \quad \text{vec}(\mathbf{XB}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{I}_m) \text{vec}(\mathbf{X})$$

- **Reihenfolge merken:**

$\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}$ und $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}^\top$ haben *immer* dieselbe Größe — ein Vertauschen fällt nicht durch unpassende Formate auf!

Bsp: vec-Trick löst die Sylvester-Gleichung

Gesucht:

$\mathbf{X} \in \mathbb{R}^{2 \times 2}$ mit $\mathbf{A}\mathbf{X} + \mathbf{X}\mathbf{B} = \mathbf{C}$ (z.B. stationäre Kovarianzen von Zeitreihenmodellen: Lyapunov-Gleichung)

vec-Trick auf beide Summanden ($m = n = 2$):

$$\text{vec}(\mathbf{A}\mathbf{X} + \mathbf{X}\mathbf{B}) = (\mathbf{I}_2 \otimes_{\mathbf{K}} \mathbf{A} + \mathbf{B}^{\top} \otimes_{\mathbf{K}} \mathbf{I}_2) \text{vec}(\mathbf{X}) = \text{vec}(\mathbf{C})$$

$\implies$ ein ganz normales 4×4 -LGS für $\text{vec}(\mathbf{X})$:

```
1 A <- matrix(c(3, 1, 0, 2), 2, 2)
2 B <- matrix(c(0, 1, 1, 1), 2, 2)
3 C <- matrix(c(1, 2, 3, 4), 2, 2)
4 # Systemmatrix des LGS via Kroneckerprodukte:
5 M <- kronecker(diag(2), A) + kronecker(t(B), diag(2))
6 X <- matrix(solve(M, as.vector(C)), 2, 2)
7 A %*% X + X %*% B          # = C ?
8 all.equal(A %*% X + X %*% B, C)
```

```
      [,1] [,2]
[1,]     1     3
[2,]     2     4
[1] TRUE
```

- Ohne vec-Trick keine LGS-Form für unsere Löser aus Kap. 5 — mit: `solve()` genügt. (Für große Matrizen gibt es effizientere spezialisierte Sylvester-Löser.)

Tensorprodukt von Vektorräumen

Tensorprodukt von Vektorräumen

Def.: Tensorprodukt von Vektorräumen

Seien V und W zwei Vektorräume. Dann ist das **Tensorprodukt** $V \otimes W$ definiert als der Vektorraum

$$V \otimes W = \text{span}\{\boldsymbol{v} \otimes \boldsymbol{w} : \boldsymbol{v} \in V, \boldsymbol{w} \in W\},$$

wobei $(\boldsymbol{v}, \boldsymbol{w}) \mapsto \boldsymbol{v} \otimes \boldsymbol{w}$ eine bilineare Abbildung ist.

- **Bilinearität** bedeutet: $(c\boldsymbol{v}) \otimes \boldsymbol{w} = c(\boldsymbol{v} \otimes \boldsymbol{w}) = \boldsymbol{v} \otimes (c\boldsymbol{w})$, $(\boldsymbol{v}_1 + \boldsymbol{v}_2) \otimes \boldsymbol{w} = \boldsymbol{v}_1 \otimes \boldsymbol{w} + \boldsymbol{v}_2 \otimes \boldsymbol{w}$ und $\boldsymbol{v} \otimes (\boldsymbol{w}_1 + \boldsymbol{w}_2) = \boldsymbol{v} \otimes \boldsymbol{w}_1 + \boldsymbol{v} \otimes \boldsymbol{w}_2$

Beispiel:

- Sei $V = \mathbb{R}^m$, $W = \mathbb{R}^n$ und $\otimes$ das äußere Produkt. Dann ist $V \otimes W = \mathbb{R}^m \otimes \mathbb{R}^n = \mathbb{R}^{m \times n}$.

Beispiel:

- Seien $V_1 = \mathbb{R}^m$, $V_2 = \mathbb{R}^n$, $V_3 = \mathbb{R}^q$ und $\otimes$ das Tensorprodukt. Dann ist

$$V_1 \otimes V_2 \otimes V_3 = \mathbb{R}^m \otimes \mathbb{R}^n \otimes \mathbb{R}^q = (\mathbb{R}^m \otimes \mathbb{R}^n) \otimes \mathbb{R}^q = \mathbb{R}^{m \times n} \otimes \mathbb{R}^q = \mathbb{R}^{m \times n \times q}.$$

Bsp: $V \otimes W$ vs. $v \otimes w$

Wichtige Unterscheidung:

- $v \otimes w$: Ein einzelnes Tensorprodukt (Rang-1-Matrix für $v, w \neq 0$)
- $V \otimes W$: Der **gesamte Vektorraum**, der von allen solchen Produkten erzeugt wird

Beispiel: Sei $V = W = \mathbb{R}^2$

- $A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ **nicht** als einzelnes Tensorprodukt $v \otimes w$ von $v, w \in \mathbb{R}^2$ darstellbar (Rang 2 $\neq$ 1).
- **Aber:** $A \in V \otimes W = \mathbb{R}^{2 \times 2}$:

$$\begin{aligned} A &= e_1 \otimes e_1 + e_2 \otimes e_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \checkmark \end{aligned}$$

Allgemein:

Jede Matrix $A \in \mathbb{R}^{m \times n}$ kann als Summe von höchstens $\min(m, n)$ Tensorprodukten geschrieben werden:
 $A = \sum_{i=1}^r \sigma_i u_i \otimes v_i$ mit $r = \text{rank}(A) \rightarrow$ **Singulärwertzerlegung!**

Tensorproduktbasis für Vektorräume

Theorem

Sind $\mathcal{B}_V = \{\mathbf{v}_1, \dots, \mathbf{v}_m\}$ und $\mathcal{B}_W = \{\mathbf{w}_1, \dots, \mathbf{w}_n\}$ Basen von V bzw. W , dann ist

$$\mathcal{B}_{V \otimes W} = \{\mathbf{v} \otimes \mathbf{w} : \mathbf{v} \in \mathcal{B}_V, \mathbf{w} \in \mathcal{B}_W\}$$

eine Basis von $V \otimes W$ und wird **Tensorproduktbasis** genannt. Insbesondere gilt

$$V \otimes W = \text{span}(\mathcal{B}_{V \otimes W}) = \left\{ \sum_{i=1}^m \sum_{j=1}^n c_{i,j} \mathbf{v}_i \otimes \mathbf{w}_j : c_{1,1}, \dots, c_{m,n} \in \mathbb{R} \right\}$$

und $\dim(V \otimes W) = \dim(V) \dim(W)$.

Bsp: Tensorproduktbasis für bivariate Polynome

Univariate Basis:

Raum der Polynome vom Grad ≤ 1 auf $[0, 1]$: $\mathcal{P}_1 = \{p(t) = a_0 + a_1 t : a_0, a_1 \in \mathbb{R}\}$ mit Basis $\mathcal{B} = \{b_1, b_2\}$, $b_1(t) = 1, b_2(t) = t$.

Tensorproduktbasis für bivariate Funktionen $f: [0, 1] \times [0, 1] \rightarrow \mathbb{R}$:

Basisfunktionen $\phi_{jk} = b_j \otimes b_k$ mit $(b_j \otimes b_k)(x, y) = b_j(x) \cdot b_k(y)$, also

$$\begin{aligned} \phi_{11}(x, y) &= b_1(x)b_1(y) = 1 & ; & & \phi_{12}(x, y) &= b_1(x)b_2(y) = y \\ \phi_{21}(x, y) &= b_2(x)b_1(y) = x & ; & & \phi_{22}(x, y) &= b_2(x)b_2(y) = xy \end{aligned}$$

Approximation einer bivariaten Funktion:

Z.B. $f(x, y) = 2 - x + 3y + 5xy = 2\phi_{11}(x, y) + 3\phi_{12}(x, y) - 1\phi_{21}(x, y) + 5\phi_{22}(x, y)$

Dimension: $\dim(\mathcal{P}_1 \otimes \mathcal{P}_1) = 2 \cdot 2 = 4 \checkmark$

Allgemein: Für k -variate Tensorprodukt-Polynome vom Grad $\leq d$ in jeder Variablen (univariat:

$$\mathcal{P}_d = \left\{ p(t) = \sum_{r=0}^d a_r t^r : a_r \in \mathbb{R} \right\} : \dim(\mathcal{P}_d^{\otimes k}) = (d+1)^k$$

Anwendung in der Praxis:

- **Statistik:** Tensorprodukt-Splines für multivariate Regression (GAMs) — ausführlich im Kapitel *Funktionsapproximation*
- **Finite-Elemente-Methoden (FEM):** Approximation von PDEs mit tensorbasierten Polynomen
- **Numerik:** Sparse Grids vermeiden die exponentielle Anzahl $(d+1)^k$ durch geschickte Auswahl

Bsp: Tensorproduktbasis für bivariate Polynome

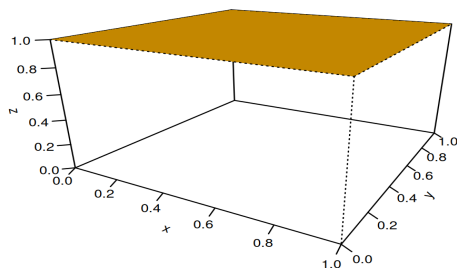
Tensorproduktbasis

$$P_1 \times P_1$$

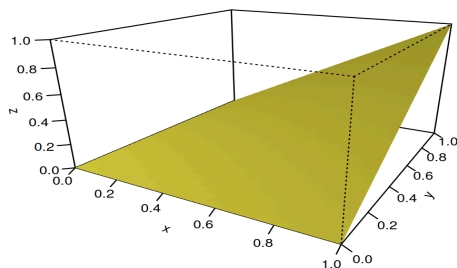
Univariate Basen:

$$B_x = \{1, x\}$$

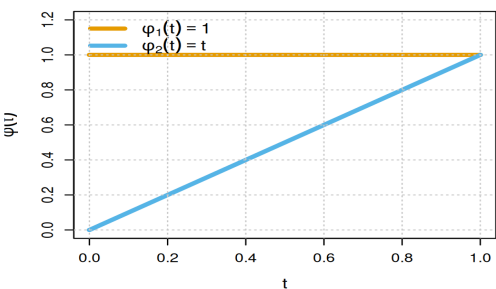
$$\varphi_1 = 1 \times 1$$



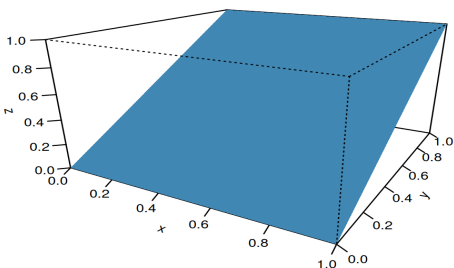
$$\varphi_4 = x \times y$$



Univariate Basen



$$\varphi_2 = 1 \times y$$



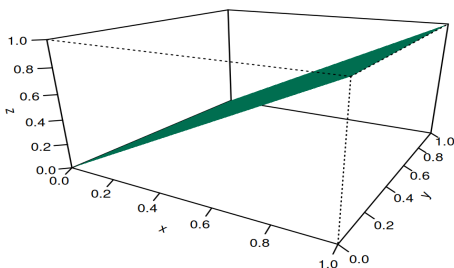
$$B_y = \{1, y\}$$

Tensorprodukt:

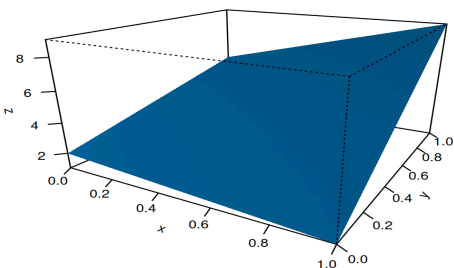
$$B_x \times B_y$$

erzeugt 4 bivariate

$$\varphi_3 = x \times 1$$



Beispiel: $f(x, y) = 2 - x + 3y + 5xy$



Anwendung: Tensorprodukt-Splines — Designmatrix

Vorgriff aufs Kapitel *Funktionsapproximation*: flexible univariate Funktionen via **B-Spline**-Basis $B_1, \dots, B_K$; bivariat via **Tensorproduktbasis** $\phi_{jk}(x, y) = B_j(x)B_k(y)$.

- Auswertung von $f(x, y) = \sum_{j,k} c_{jk} \phi_{jk}(x, y)$ auf einem **Gitter** aus x - und y -Werten:

$$\underbrace{\mathbf{F}}_{f(x_i, y_l)} = \mathbf{B}_x \mathbf{C} \mathbf{B}_y^\top$$

mit univariaten Basismatrizen $\mathbf{B}_x \in \mathbb{R}^{n_x \times K}$,
 $(\mathbf{B}_x)_{ij} = B_j(x_i)$ (analog $\mathbf{B}_y$)

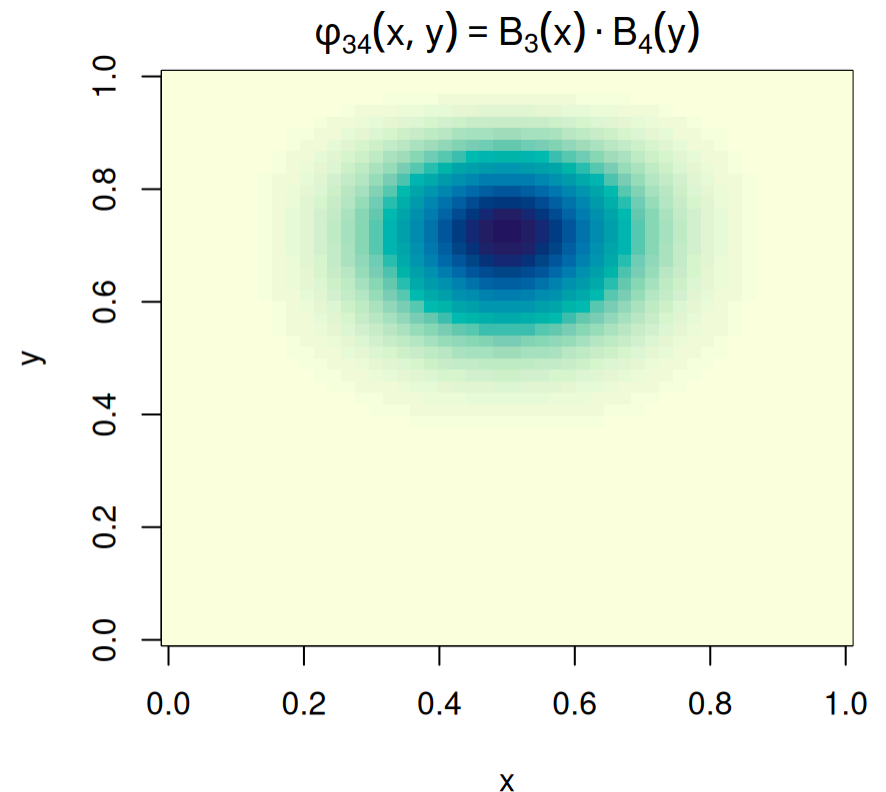
- **vec-Trick!**

$$\text{vec}(\mathbf{F}) = (\mathbf{B}_y \otimes_K \mathbf{B}_x) \text{vec}(\mathbf{C})$$

$\implies$ Designmatrix des Gitters ist das **Kroneckerprodukt**
 $\mathbf{B} = \mathbf{B}_y \otimes_K \mathbf{B}_x$

```
1 library(splines)
2 x <- seq(0, 1, length.out = 50)
3 Bx <- bs(x, df = 6) # B-Spline-Basis
4 By <- bs(x, df = 6)
5 B <- kronecker(By, Bx)
6 dim(B) # 2500 Gitterpunkte, 6*6 Koeffizienten
```

[1] 2500 36



Eine bivariate Basisfunktion = äußeres Produkt zweier univariater B-Splines. $\rightarrow$ Kap. *Funktionsapproximation I/II*: gute univariate Basen $\otimes$ -kombinieren!

Self-Check & Zusammenfassung

Kernkonzepte & Anwendungen

- **Tensoren**: höher-dimensionale Verallgemeinerung von Vektoren und Matrizen
 - Extrem wichtig für Deep Learning, Bild-/Videobearbeitung, moderne Statistik
- **Multilinearität** vs Linearität:
 - Linear: $f(a\mathbf{v} + b\mathbf{w}) = af(\mathbf{v}) + bf(\mathbf{w})$ vs. Multilinear:
 $f(\mathbf{v}_1, \dots, a\mathbf{v}_i + b\mathbf{w}_i, \dots, \mathbf{v}_n) = af(\mathbf{v}_1, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n) + bf(\mathbf{v}_1, \dots, \mathbf{w}_i, \dots, \mathbf{v}_n)$
 - Bsp: Matrixmultiplikation bilinear aber nicht linear.
- **Äußeres Produkt**: Erzeugt (Rang-1-)Matrizen aus Vektoren ($\rightarrow$ **SVD**)
- **Tensorprodukt**: Verallgemeinerung des äußeren Produkts auf Tensoren höherer Stufe
- **Kroneckerprodukt** $\otimes_{\mathbb{K}}$: Spezielle Variante des Tensorprodukts von Matrizen ($\otimes$)
 - **vec-Trick** $\text{vec}(\mathbf{AXB}) = (\mathbf{B}^{\top} \otimes_{\mathbb{K}} \mathbf{A})\text{vec}(\mathbf{X})$: Matrixgleichungen (Sylvester/Lyapunov) $\rightarrow$ gewöhnliche LGS
 - Statistik: separierbare/Kronecker-strukturierte Kovarianzen, blockdiagonale Matrizen
- *Tensorprodukte bauen komplexe Räume auf:*
 - Konstruiere multivariate (Funktions-)Räume aus univariaten — Designmatrix auf Gittern = Kroneckerprodukt $\rightarrow$ **Funktionsapproximation**, (Höherdimensionale) **Splines**

Nächste Vorlesungen:

Fortgeschrittene Differentialrechnung

Self-Check

Frage 1: Daten an $m = 10$ Orten zu $n = 20$ Zeitpunkten. Eine unstrukturierte Kovarianzmatrix des 200-dimensionalen Datenvektors hat $\frac{200 \cdot 201}{2} = 20100$ freie Parameter. Wie viele Parameter haben die beiden Faktoren Σ_T, Σ_S des separierbaren Modells $\Sigma = \Sigma_T \otimes_K \Sigma_S$ zusammen?

- a) 265 b) 11550 c) 20100 d) 200

Frage 2: Sei $f: \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ bilinear. Was ist $f(2v, 3w)$?

- a) $2f(v, w)$ b) $5f(v, w)$ c) $6f(v, w)$ d) $f(v, w)$

Frage 3: Der vec-Trick besagt $\text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = ?$

- a) $(\mathbf{A} \otimes_K \mathbf{B}) \text{vec}(\mathbf{X})$ b) $(\mathbf{B}^\top \otimes_K \mathbf{A}) \text{vec}(\mathbf{X})$ c) $(\mathbf{B} \otimes_K \mathbf{A}^\top) \text{vec}(\mathbf{X})$ d) $(\mathbf{A} \otimes_K \mathbf{B}^\top) \text{vec}(\mathbf{X})$

Anhang

Anwendung: RGB-Bilder als Tensoren

Beispiel 1: RGB-Bild

- Farbbild der Größe 224×224 Pixel wird als Tensor $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 3}$ dargestellt:
 - $\mathbf{I}_{i,j,1}$: Rot-Intensität an Position (i, j) (Wert zwischen 0 und 255)
 - $\mathbf{I}_{i,j,2}$: Grün-Intensität an Position (i, j)
 - $\mathbf{I}_{i,j,3}$: Blau-Intensität an Position (i, j)
- Einträge: $224 \times 224 \times 3 \approx 150 \cdot 10^3$ Werte pro Bild



$\approx$



↔ zurück: „Tensoren in Deep Learning“

Anwendung: Batch-Tensoren in CNNs

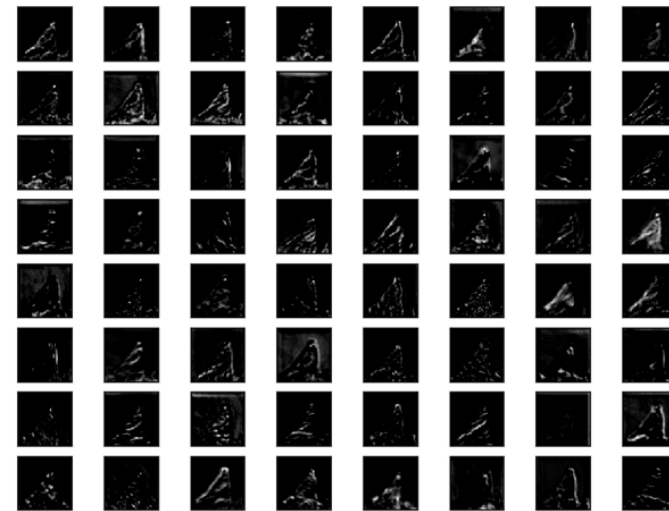
Beispiel 2: Batch von Bildern (Convolutional Neural Networks) (CNNs)

- CNN verarbeitet typischerweise mehrere Bilder gleichzeitig: $\mathbf{B} \in \mathbb{R}^{32 \times 224 \times 224 \times 3}$
 1. Dimension (Batch-Size): 32 Bilder gleichzeitig
 2. Dimension (Höhe): 224 Pixel
 3. Dimension (Breite): 224 Pixel
 4. Dimension (Kanäle): 3 Farbkanäle (RGB)
- Einträge: $32 \times 224 \times 224 \times 3 \approx 4.8 \cdot 10^6$ Werte

Anwendung: Feature Maps in CNNs

Beispiel 3: Feature Maps in CNNs

- Convolutional Layer $c: \mathbb{R}^{32 \times 224 \times 224 \times 3} \rightarrow \mathbb{R}^{32 \times 56 \times 56 \times 64}$
- Reduziert (üblicherweise) räumliche Auflösung ($224 \times 224 \rightarrow 56 \times 56$)
- Erhöht Anzahl der “Kanäle” (*features*) ($3 \rightarrow 64$)



Feature Map: Block 3 des VGG16-Modells; Quelle: machinelearningmastery.com

↔ zurück: „Tensoren in Deep Learning“

Universelle Eigenschaft des Tensorprodukts

- Das **Tensorprodukt** $V \otimes W$ hat eine *universelle Eigenschaft*: **Jede** bilineare Abbildung $f: V \times W \rightarrow Z$ kann eindeutig als lineare Abbildung $\tilde{f}: V \otimes W \rightarrow Z$ faktorisiert werden:

$$f(\boldsymbol{v}, \boldsymbol{w}) = \tilde{f}(\boldsymbol{v} \otimes \boldsymbol{w}) \quad \text{für alle } \boldsymbol{v} \in V, \boldsymbol{w} \in W$$

- Diese Eigenschaft macht das Tensorprodukt zur “natürlichen” Konstruktion für die Linearisierung bilinearer Abbildungen.